

Aim : Write programs to implement the following Substitution Cipher Techniques: - Caesar Cipher - Monoalphabetic Cipher

Code : 1. Caesar Cipher

```
package javaapplication9;
import java.util.*;

class JavaApplication9{
    public static StringBuffer encryption(String text,int k){
        StringBuffer result=new StringBuffer();
        for(int i=0;i<text.length();i++){
            if(Character.isUpperCase(text.charAt(i))){
                char ch=(char)(((int)text.charAt(i)+k-65)%26+65);
                result.append(ch);
            }

            else{
                char ch=(char)(((int)text.charAt(i)+k-97)%26+97);
                result.append(ch);
            }
        }
        return result;
    }
    public static StringBuffer decryption(String text, int
k){
        StringBuffer result1 = new StringBuffer();

        for(int i = 0; i < text.length(); i++){
            if(Character.isUpperCase(text.charAt(i))){
                char ch = (char)(((text.charAt(i) - 65 - k + 26)
% 26) + 65);
                result1.append(ch);
            }
        }
    }
}
```

```
        else{
            char ch = (char)(((text.charAt(i) - 97 - k + 26)
% 26) + 97);
            result1.append(ch);
        }
    }
    return result1;
}
}
public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter string: ");
    String s = sc.next();
```

```
    System.out.print("Enter key: ");
    int k = sc.nextInt();

    k = k % 26;

    String str = encryption(s, k).toString();

    System.out.println("Cipher Text: " + str);
    System.out.println("Plain Text: " + decryption(str,
k));
}
}
```

OUTPUT :

```
Enter string: Akash
Enter key: 3
Cipher Text: Dndvk
Plain Text: Akash
```

2. Monoalphabetic Cipher

```
package javaapplication9;
import java.util.*;
import java.io.*;
public class MONO {

    public static void main(String[]args){
        String
key="ZXCBMNASDFGHJKLQWERTYUIOP
V";

        Scanner sc=new Scanner(System.in);
        System.out.println("Enter Plain Text");
        String pt=sc.next().toUpperCase();
        String ct = encrypt(pt, key);
        String pt1=decrypt(ct,key);
        System.out.println("Encrypted "+ct);
        System.out.println("Decrypted "+pt1);

    }

    public static String encrypt(String s,
String k){
        StringBuffer sb =new StringBuffer(s);
        for(int i=0;i<sb.length();i++){
            int idx;
            char c;
            idx=sb.charAt(i)-65;
            c=k.charAt(idx);
            sb.setCharAt(i, c);
        }
    }
}
```

```
String encryptedText = sb.toString();
return encryptedText;

}

public static String decrypt(String s,String
key){
    StringBuffer sb =new StringBuffer(s);
    for (int i=0;i<sb.length();i++){
        int idx;
        char c;
        c=sb.charAt (i );
```

```
        idx=key.indexOf(c);

        sb.setCharAt(i,(char)(idx+65));
    }
    return sb.toString();

}
}
```

OUTPUT :

```
Enter Plain Text
AKash
Encrypted ZGZRS
Decrypted AKASH
```

Roll no : 10

Practical 2

Aim : Write programs to implement the following Substitution Cipher Techniques

Codes :

VERNAM CIPHER

```
package javaapplication9;
import java.util.Scanner;

public class vernamCipher {
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.println("The length of the plain
text and the key must be same\n");
        System.out.println("Enter plain text: ");
        String pt = sc.next().toUpperCase().trim();
        System.out.println("Enter key: ");
        String key = sc.next().toUpperCase().trim();
        boolean comparison =
compareKeyPt(pt,key);
        if (comparison){
            System.out.println("Correct");
            StringBuffer ct=Encryption(pt,key);
            System.out.println("Encryption:"+ct);
            StringBuffer pt1 =
Decryption(ct.toString() , key);
            System.out.println("Decryption:"+pt1);
        }
    }
}
```

```
        else{
            System.out.println("The length of pt and
key must be same");
        }
    }
    public static boolean compareKeyPt(String pt
, String key){
        if (key.length()==pt.length()){
            return true;
        }
        else{
            return false;
        }
    }
    public static StringBuffer Encryption (String pt
, String key){
        StringBuffer result = new StringBuffer();
```

```
        for (int i=0;i<pt.length();i++){
            char ch =
(char)(((int)(pt.charAt(i)^key.charAt(i))+97);
            result.append(ch);
        }
        return result;
    }
}
```

```
    public static StringBuffer Decryption (String ct
, String key){
        StringBuffer result = new StringBuffer();
        for (int i =0;i<ct.length();i++){
            char ch =
(char)(((int)(ct.charAt(i)-97)^key.charAt(i));
            result.append(ch);
        }
        return result;
    }
}
```

OUTPUT :

```
Enter plain text:
AKASH
Enter key:
aaabbbbd
The length of pt and key must be same
```

```
Enter plain text:
AKASH
Enter key:
AABBC
Correct
Encryption:akdr1
Decryption:AKASH
```

Roll no : 10

Practical 3

AIM : Write programs to implement the following Transposition Cipher Techniques:

Code : KEY : 2

```
import java.util.*;

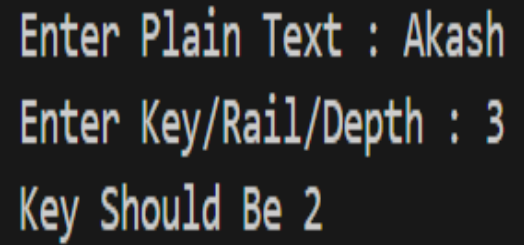
public class Rail_Fence {

    public static StringBuffer
    encryption(String PT,int Key){
        StringBuffer result1 = new
        StringBuffer();
        StringBuffer result2 = new
        StringBuffer();
        StringBuffer result = new
        StringBuffer();
        for(int i=0;i<PT.length();i++){
            if(i%2==0){
                result1.append(PT.charAt(i));
            }
            else{
                result2.append(PT.charAt(i));
            }
        }
        result.append(result1);
        result.append(result2);
        return result;
    }
}
```

```
public static void main(String[] args){
    Scanner sc = new
    Scanner(System.in);
    System.out.print("Enter Plain Text :
    ");
    String PT =
    sc.nextLine().toLowerCase();
    System.out.print("Enter
    Key/Rail/Depth : ");
    int Key = sc.nextInt();

    if(Key==2){
        String a =
        encryption(PT,Key).toString();
        System.out.println("Encrypted Text
        : "+a);
    }
    else {
        System.out.println("Key Should Be
        2");
    }
}
```

OUTPUT :



```
Enter Plain Text : Akash
Enter Key/Rail/Depth : 3
Key Should Be 2
```

KEY = Custom

```
import java.util.*;

public class RAILFANCE_CUSTOM
{
    public static void main (String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter plain text: ");
        String pt = sc.nextLine();
        System.out.println("Enter key : ");
        pt=pt.replaceAll("\\s+", "");
        int key = sc.nextInt();

        char[][] rail = new char[key][pt.length()];

        for(int i=0;i<key;i++){
            for(int j=0;j<pt.length();j++){
                rail[i][j]='\n';
            }
        }
        int row = 0;
        boolean down = true;

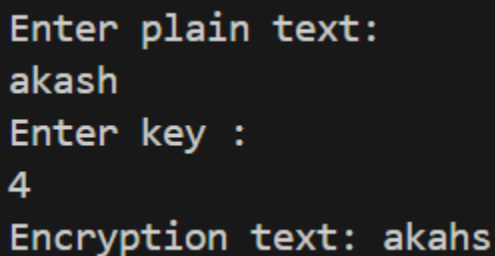
        for(int i=0;i<pt.length();i++){
            rail[row][i] = pt.charAt(i);
```

```
            if (row == 0)
                down = true;
            else if (row == key-1)
                down = false;

            if(down)
                row++;
            else
                row--;
        }
        String cipher = "";
        for(int i=0;i<key;i++){
            for(int j=0;j<pt.length();j++){
                if(rail[i][j]!='\n')
                    cipher += rail[i][j];
            }
        }
        System.out.println("Encryption text:
        "+cipher);

        sc.close();
    }
}
```

OUTPUT :



```
Enter plain text:
akash
Enter key :
4
Encryption text: akahs
```

Aim : Write programs to implement the following Transposition Cipher Techniques

Code :

```
import java.util.*;

public class SimpleColumnar {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.next();

        System.out.print("Enter Number of
Columns: ");
        int col = sc.nextInt();

        int row = (text.length() + col - 1) / col;

        char[][] matrix = new char[row][col];

        int k = 0;

        // Fill row-wise
        for (int i = 0; i < row; i++) {
            for (int j = 0; j < col; j++) {
                if (k < text.length())
                    matrix[i][j] = text.charAt(k++);
                else
                    matrix[i][j] = 'X';
            }
        }
    }
}
```

OUTPUT :

```
Enter Plain Text: Akash
Enter Number of Columns: 3

Matrix:
A k a
s h X

Cipher Text: AskhaX
```

```
// Display matrix
System.out.println("\nMatrix:");
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}

// Read column-wise
System.out.print("\nCipher Text: ");
for (int j = 0; j < col; j++) {
    for (int i = 0; i < row; i++) {
        System.out.print(matrix[i][j]);
    }
}

sc.close();
}
```

Roll no : 10

Practical 5

Aim : Write program to encrypt and decrypt strings using DES & AES Algorithm

Code :

```
import java.util.Base64;
import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;

public class DES_AES {
    static SecretKey generateKey(String algorithm) throws Exception {
        KeyGenerator kg = KeyGenerator.getInstance(algorithm);
        return kg.generateKey();
    }

    static String encrypt(String text, SecretKey key, String algorithm)
    throws Exception {
        Cipher cipher = Cipher.getInstance(algorithm);

        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encrypted = cipher.doFinal(text.getBytes());

        return Base64.getEncoder().encodeToString(encrypted);
    }

    static String decrypt(String encryptedText, SecretKey key, String
    algorithm) throws Exception {
        Cipher cipher = Cipher.getInstance(algorithm);

        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] encrypted = Base64.getDecoder().decode(encryptedText);

        byte[] decrypted = cipher.doFinal(encrypted);

        return new String(decrypted);
    }
}
```

```
public static void main(String[] args) throws Exception {

    Scanner sc = new Scanner(System.in);
    System.out.println("1. DES");
    System.out.println("2. AES");
    System.out.print("Choose Algorithm: ");

    int choice = sc.nextInt();
    sc.nextLine();

    String algorithm;

    if (choice == 1) {
        algorithm = "DES";
    } else if (choice == 2) {
        algorithm = "AES";
    } else {
        System.out.println("Invalid Choice");
        sc.close();
        return;
    }

    System.out.print("Enter Plain Text: ");
    String text = sc.nextLine();
    SecretKey key = generateKey(algorithm);
    String encrypted = encrypt(text, key, algorithm);
    String decrypted = decrypt(encrypted, key, algorithm);

    System.out.println("\nAlgorithm: " + algorithm);
    System.out.println("Encryption: " + encrypted);
    System.out.println("Decryption: " + decrypted);
    sc.close();
}
```

OUTPUT :

1) DES

2) AES

```
1. DES
2. AES
Choose Algorithm: 1
Enter Plain Text: Akash

Algorithm: DES
Encryption: bZ9Xq19n00U=
Decryption: Akash
```

```
1. DES
2. AES
Choose Algorithm: 2
Enter Plain Text: Akash

Algorithm: AES
Encryption: j9szgC8hXk8ZLJf1cSeixA==
Decryption: Akash
```

Aim : Write a program to implement RSA algorithm to perform encryption/ decryption of a given string.

Code :

```
import java.math.BigInteger;
import java.util.Scanner;

public class RSA {

    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter p: ");
        int p = sc.nextInt();
        System.out.print("Enter q: ");
        int q = sc.nextInt();
        int n = p * q;
        int phi = (p - 1) * (q - 1);
        // to calculate e
        int e = 2;
        while (e < phi) {
            int a = e;
            int b = phi;
            // to calculate gcd
            while (b != 0) {
                int temp = b;
                b = a % b;
                a = temp;
            }
            if (a == 1)
                break;
            e++;
        }
        // to calculate d
        int d = 1;
        while ((d * e) % phi != 1) {
            d++;
        }

        System.out.println("n = " + n);
        System.out.println("phi = " + phi);
        System.out.println("e = " + e);
        System.out.println("d = " + d);
```

```
System.out.print("Enter message: less than " + n + " ");
int message = sc.nextInt();

BigInteger M = BigInteger.valueOf(message);
BigInteger E = BigInteger.valueOf(e);
BigInteger N = BigInteger.valueOf(n);
BigInteger D = BigInteger.valueOf(d);

BigInteger encrypted = M.modPow(E, N);

BigInteger decrypted = encrypted.modPow(D, N);

System.out.println("Encrypted = " + encrypted);
System.out.println("Decrypted = " + decrypted);
sc.close();
}
```

OUTPUT :

```
Enter p: 13
Enter q: 29
n = 377
phi = 336
e = 5
d = 269
Enter message: less than 377 300
Encrypted = 66
Decrypted = 300
```

Aim : Write a program to implement the Diffie-Hellman Key Agreement algorithm to generate symmetric keys.

Code :

<pre>import java.util.Scanner; public class DiffieHellman { // Calculate (base ^ power) % q static int power(int base, int power, int q) { int result = 1; for (int i = 1; i <= power; i++) { result = (result * base) % q; } return result; } public static void main(String[] args) { Scanner sc = new Scanner(System.in); // Public values System.out.print("Enter q: "); int q = sc.nextInt(); System.out.print("Enter alpha: "); int alpha = sc.nextInt();</pre>	<pre>// Private keys System.out.print("Enter private key XA: "); int xa = sc.nextInt(); System.out.print("Enter private key XB: "); int xb = sc.nextInt(); // Public keys int ya = power(alpha, xa, q); int yb = power(alpha, xb, q); // Secret keys int keyA = power(yb, xa, q); int keyB = power(ya, xb, q); System.out.println("\nPublic Key of A = " + ya); System.out.println("Public Key of B = " + yb); System.out.println("Secret Key of A = " + keyA); System.out.println("Secret Key of B = " + keyB); sc.close(); } }</pre>
---	---

OUTPUT :

```
Enter q: 7
Enter alpha: 23
Enter private key XA: 13
Enter private key XB: 11

Public Key of A = 2
Public Key of B = 4
Secret Key of A = 4
Secret Key of B = 4
```

Aim : Write a program to implement the MD5 algorithm compute the message digest.

Input :

<pre>import java.security.MessageDigest; import java.util.HexFormat; public class mds { public static String getHash(byte[] inputBytes, String algorithm) { String hashvalue = ""; try { MessageDigest messageDigest = MessageDigest.getInstance(algorithm); messageDigest.update(inputBytes); byte[] digestedBytes = messageDigest.digest();</pre>	<pre>// Convert bytes to lowercase hexadecimal hashvalue = HexFormat.of().formatHex(digestedBytes); } catch (Exception e) { e.printStackTrace(); } return hashvalue; } public static void main(String[] args) { String somestring = "this is some string"; System.out.println(getHash(somestring.getBytes(), "SHA-256"));}}</pre>
---	---

OUTPUT :

```
b5342e4a43f3a44c22cc4f5a12f5cbb46de0610d585abed6b09b6465a4d35c88
```